ПРОГРАММНАЯ ДОКУМЕНТАЦИЯ
НА ПРОГРАММНЫЙ МОДУЛЬ "PC-HELPER"
,
РАЗРАБОТАННЫЙ НА ОСНОВЕ
ЯЗЫКА ПРОГРАММИРОВАНИЯ PYTHON

ЮЖНО-САХАЛИНСК
2026

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ....3
2. ЧАСТНОЕ ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПОДСИСТЕМУ "PC-Helper"....5
2.1. Назначение разработки....5
2.2. Основание для разработки....5
2.3. Требования к программному обеспечению....5
2.3.1. Требования к инструментальной среде....5
2.3.2. Требования к пользовательскому интерфейсу....5
2.3.3. Требования к реализуемым функциям....6
2.4. Требования к надежности....6
3. СТРУКТУРНАЯ СХЕМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ....7
3.1. Общая структурная схема....7
4. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ....8
5. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВХОДНЫХ ПЕРЕМЕННЫХ....12
6. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ....13
7. ПОЯСНЕНИЯ К ИСПОЛЬЗУЕМОЙ НОТАЦИИ....15
8. СИСТЕМОНЕЗАВИСИМАЯ ЧАСТЬ....16
8.1. Главный управляющий класс SystemControlPanel....16
8.1.1. Полный код класса....16
8.1.2. Вспомогательные функции (файл запуска)....24
9. ТЕКСТЫ СИСТЕМОНЕЗАВИСИМОЙ ЧАСТИ....25
9.1. Модуль инициализации (файл main.py)....25
9.2. Модуль главного класса (файл system_control.py)....26
9.3. Модуль проверки прав (файл admin_check.py)....27
9.4. Модуль системных действий Windows (файл windows_actions.py)....27
9.5. Модуль системных действий macOS (файл mac_actions.py)....28
9.6. Модуль системных действий Linux (файл linux_actions.py)....28
9.7. Модуль веб-действий (файл web_actions.py)....29
9.8. Модуль пользовательского интерфейса (файл ui_components.py)....30

1. ВВЕДЕНИЕ

Настоящий документ представляет собой программную документацию на программный продукт PC-Helper, разработанный в рамках выполнения проектного задания по дисциплине "Разработка программных модулей". Программа PC-Helper представляет собой приложение с графическим интерфейсом, предназначенное для обеспечения быстрого и удобного доступа пользователя к наиболее востребованным системным функциям и приложениям операционной системы. К таким функциям относятся открытие веб-браузера и популярных интернет-ресурсов, запуск системного проводника, текстового редактора и калькулятора, а также выполнение критических операций управления питанием компьютера, таких как выключение и перезагрузка. Актуальность разработки подобного программного инструмента обусловлена необходимостью оптимизации повседневной деятельности пользователя за счет сокращения времени на выполнение рутинных операций. Объединение часто используемых функций в едином, интуитивно понятном интерфейсе позволяет повысить эффективность работы и снизить когнитивную нагрузку на пользователя.

Основной целью создания программного продукта PC-Helper является разработка функционального, надежного и кроссплатформенного приложения, демонстрирующего применение современных подходов к проектированию событийно-управляемых программ. В процессе разработки решались задачи проектирования удобного графического интерфейса, реализации механизмов взаимодействия с различными операционными системами (Windows, macOS, Linux), обеспечения безопасного выполнения привилегированных операций с контролем прав доступа, а также создания системы обработки исключительных ситуаций и протоколирования действий. Программа реализована на языке Python с использованием стандартной библиотеки Tkinter для построения графического интерфейса и встроенных модулей os, subprocess, platform для взаимодействия с операционной системой. Такой выбор технологий обеспечивает легкость, кроссплатформенность и простоту распространения приложения без необходимости установки дополнительных зависимостей. Данная

документация содержит полное описание функциональных характеристик, архитектуры, программных модулей и протоколов функционирования разработанного программного продукта.

2. ЧАСТНОЕ ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПОДСИСТЕМУ "PC-Helper"

2.1. Назначение разработки

Подсистема "PC-Helper" предназначена для упрощения и ускорения доступа пользователя к часто используемым системным функциям (завершение работы, перезагрузка) и приложениям (браузер, проводник, калькулятор).

2.2. Основание для разработки

Разработка осуществляется в рамках проектного задания по дисциплине “Разработка программных модулей”.

2.3. Требования к программному обеспечению

2.3.1. Требования к инструментальной среде

Программа предназначена для функционирования под управлением интерпретатора Python версии не ниже 3.6.

Для создания графического интерфейса используется стандартная библиотека tkinter.

2.3.2. Требования к пользовательскому интерфейсу

Подсистема работает как отдельное приложение (оконный процесс).

Размер окна подсистемы фиксирован и составляет 800x500 пикселей.

Окно должно быть оформлено в темной цветовой гамме для снижения нагрузки на зрение.

Элементы управления (кнопки) должны быть сгруппированы по функциональному признаку, иметь цветовую индикацию и всплывающие подсказки в виде статусной строки.

Должна быть реализована поддержка горячих клавиш для основных действий (Ctrl+B, Ctrl+Y, Esc и др.).

2.3.3. Требования к реализуемым функциям

Подсистема должна предоставлять следующие основные функции

Открытие веб-браузера (по умолчанию Google).

Открытие видеохостинга YouTube.

Открытие системного проводника в текущей директории.

Открытие текстового редактора (Блокнот / аналог).

Открытие системного калькулятора.

Открытие репозитория GitHub.

Подсистема должна предоставлять следующие функции управления питанием:

Выключение компьютера с подтверждением.

Перезагрузка компьютера с подтверждением.

Функции выключения и перезагрузки должны проверять наличие прав администратора и предупреждать пользователя об их отсутствии и о возможной потере несохраненных данных.

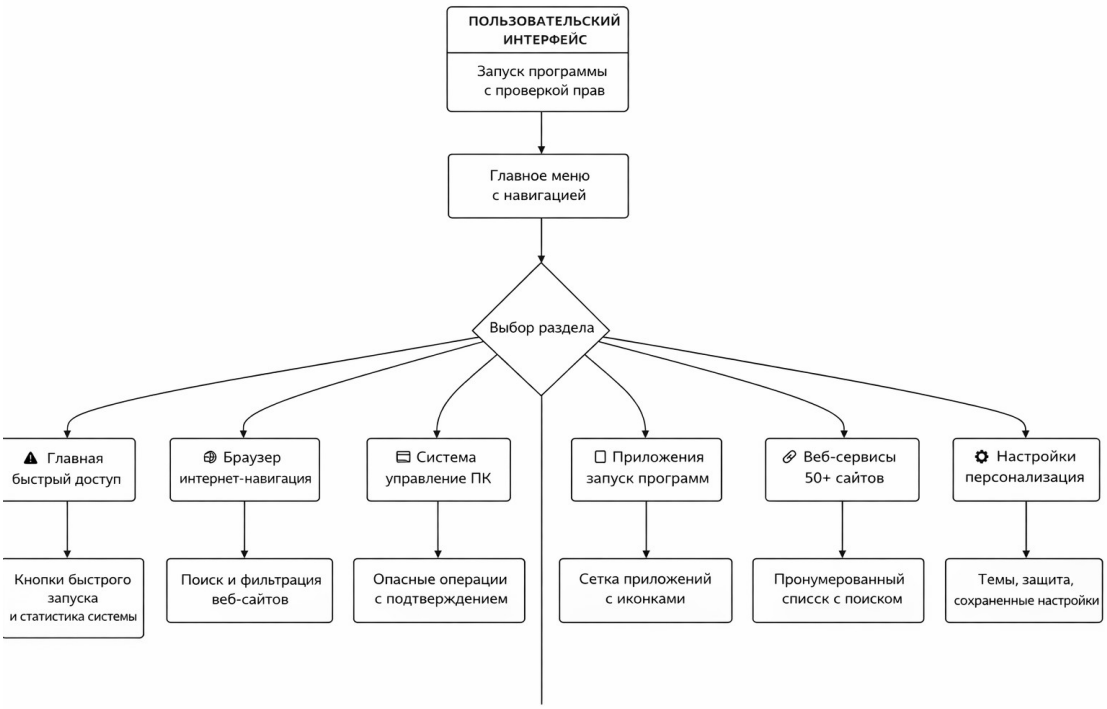
Подсистема должна отображать краткую информацию о системе (ОС, версия Python, процессор) и статусе прав администратора.

2.4. Требования к надежности

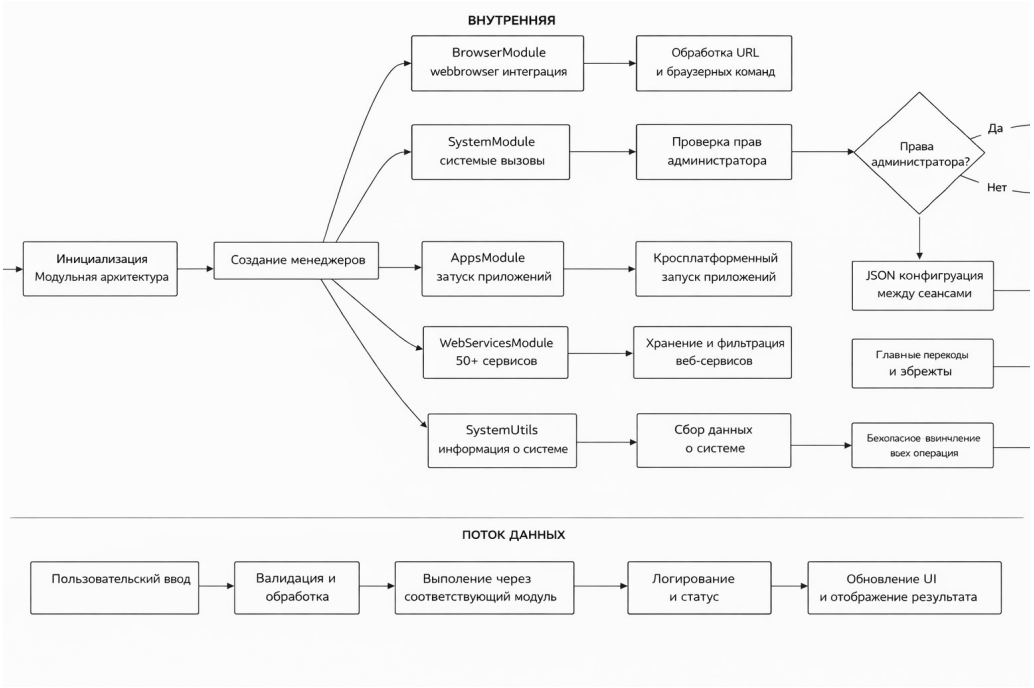
Должна быть предусмотрена защита от случайного выполнения критических операций (выключение, перезагрузка) через диалог подтверждения.

3. СТРУКТУРНАЯ СХЕМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

3.1. Общая структурная схема



3.1. Общая структурная схема



4. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ

№	Наименование	Тип	Назначение
1	SystemControlPanel	Класс	Главный класс приложения, управляющий созданием графического интерфейса и обработкой всех событий
2	Init(self, root)	Метод	Конструктор класса, инициализирует главное окно и запускает создание всех элементов интерфейса
3	setup_window(self)	Метод	Настраивает параметры главного окна (размер, заголовок, цветовую гамму)
4	create_widgets(self)	Метод	Создает все визуальные элементы интерфейса (кнопки, надписи, информационные панели)
5	create_buttons(self)	Метод	Формирует ряды кнопок с

			соответствующими командами и цветовым оформлением
6	create_system_info(self)	Метод	Создает панель с информацией о системе и статусную строку
7	setup_bindings(self)	Метод	Настраивает обработчики горячих клавиш
8	open_browser()	Метод	Открывает веб-браузер с поисковой системой Google
9	open_youtube()	Метод	Открывает видеохостинг YouTube в браузере
10	open_explorer()	Метод	Открывает системный проводник в текущей директории
11	shutdown_computer()	Метод	Выполняет выключение компьютера с подтверждением
12	restart_computer()	Метод	Выполняет перезагрузку компьютера с подтверждением
13	exit_app()	Метод	Завершает работу

			приложения
14	open_notepad()	Метод	Запускает текстовый редактор (Блокнот в Windows, TextEdit в macOS, gedit в Linux)
15	open_calculator	Метод	Запускает системный калькулятор
16	open_github()	Метод	Открывает в браузере сайт GitHub
17	is_admin()	Метод	Проверяет наличие прав администратора (root-прав) в системе
18	show_status(message)	Метод	Выводит сообщение в консоль и обновляет статусную строку
19	show_error(message)	Метод	Отображает диалоговое окно с сообщением об ошибке
20	show_warning(message)	Метод	Отображает диалоговое окно с предупреждением
21	check_admin()	Функция	Проверяет и при необходимости

			запрашивает права администратора при запуске
22	main()	Функция	Главная функция запуска приложения, создает окно и запускает цикл обработки событий

5. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВХОДНЫХ ПЕРЕМЕННЫХ

№	Переменная	Описание	Где используется
1	Статус администратора	Флаг наличия прав администратора (True/False)	В функциях shutdown_computer() для проверки возможности выполнения операций
2	Подтверждение пользователя	Результат выбора в диалоговом окне (Да/Нет)	В функциях shutdown_computer(), restart_computer(), exit_app() перед выполнением критических действий
3	Тип операционной системы	Значение, возвращаемое platform.system() (Windows/Darwin/Linux)	Во всех функциях взаимодействия с ОС для выбора правильной команды
4	Состояние порта/процесса	Статус доступности устройств (для калькулятора, для блокнота и т.д)	При запуске внешних приложений для проверки успешности выполнения

6. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ

№	Воздействие	Описание	Реализация
1	Открыть браузер	Запуск веб-браузера с переходом на Google	webbrowser.open("https://google.com")
2	Открыть YouTube	Запуск браузера с переходом на YouTube	webbrowser.open("https://youtube.com")
3	Открыть проводник	Запуск системного файлового менеджера	os.startfile(".") или subprocess.Popen(["open", "."]) или subprocess.Popen(["xdg-open", "."])
4	Выключить компьютер	Завершение работы системы с таймером 5 секунд	os.system("shutdown /s /t 5") или subprocess.Popen(["sudo", "shutdown", "-h", "now"])
5	Перезагрузить компьютер	Перезагрузка системы с таймером 5 секунд	os.system("shutdown /r /t 5") или subprocess.Popen(["sudo", "shutdown", "-r", "now"])
6	Открыть блокнот	Запуск текстового редактора	os.system("notepad") или subprocess.Popen(["open", "-a", "TextEdit"]) или subprocess.Popen(["gedit"])
7	Открыть калькулятор	Запуск системного калькулятора	os.system("calc") или subprocess.Popen(["open", "-a", "Calculator"]) или subprocess.Popen(["gnome-calculator"])
8	Открыть GitHub	Запуск	webbrowser.open("https://github.com")

		браузера с переходом на GitHub	
9	Показать информацию	Вывод информационного сообщения	messagebox.showinfo()
10	Показать предупреждение	Вывод предупреждающего сообщения	messagebox.showwarning()
11	Показать ошибку	Вывод сообщения об ошибке	messagebox.showerror()
12	Обновить статус	Изменения текста в статусной строке	status_label.config(text=message)
13	Завершить программу	Закрытие приложения	root.destroy()

7. ПОЯСНЕНИЯ К ИСПОЛЬЗУЕМОЙ НОТАЦИИ

Данная программа имеет exe файл, можно открыть кликвнув 2 раза по иконке, после этого вас попросят выдать этой программе администратора дл ятого что бы работал весь функционал программы. После этого открывается окно панели управления, на ней находятся 9 кнопок:

- Открыть браузер (при нажатии открывает ваш браузер)
- Открыть ютуб (при нажатии открывает ютуб)
- Открыть проводник (при нажатии открывает проводник)
- Выключить компьютер(при нажатии спросит дополнительное подтверждение - выключает компьюетр, только с админ правами!)
- Перезагрузить компьютер (при нажатии спросит дополнительное подтверждение - перезагружает компьютер, только с админ правами!)
- Выход из программы (при нажатии завершает программу)
- Блокнот (при нажатии открывает блокнот)
- Калькулятор (при нажатии открывает калькулятор
- GitHub (при нажатии открывает гитхаб ваш профиль)

Программа имеет графический визуал для полного удобства

8. СИСТЕМОНЕЗАВИСИМАЯ ЧАСТЬ

8.1. Главный управляющий класс SystemControlPanel

8.1.1. Полный код класса

```
class SystemControlPanel:
    def __init__(self, root):
        self.root = root
        self.setup_window()
        self.create_widgets()
        self.setup_bindings()

    def setup_window(self):
        """Настройка окна"""
        self.root.title("🚀 System Control Panel")
        self.root.geometry("800x500")

        self.bg_color = '#2b2b2b'
        self.btn_color = '#3c3c3c'
        self.text_color = '#ffffff'
        self.highlight_color = '#4a90e2'

        self.root.configure(bg=self.bg_color)

    def create_widgets(self):
        """Создание всех элементов интерфейса"""
        # Заголовок
        title_frame = tk.Frame(self.root, bg=self.bg_color)
        title_frame.pack(pady=20)

        tk.Label(title_frame, text="⚡ SYSTEM CONTROL PANEL",
                font=("Arial", 20, "bold"),
                bg=self.bg_color, fg=self.highlight_color).pack()

        tk.Label(title_frame, text="Быстрый доступ к системным функциям",
```

```
            font=("Arial", 10),
            bg=self.bg_color, fg=self.text_color).pack(pady=5)

    self.create_buttons()

    self.create_system_info()

def create_buttons(self):
    """Создание кнопок управления"""
    button_frame = tk.Frame(self.root, bg=self.bg_color)
    button_frame.pack(pady=20)

    row1 = tk.Frame(button_frame, bg=self.bg_color)
    row1.pack(pady=10)

    buttons_row1 = [
        ("🌐 Открыть браузер", self.open_browser, '#4CAF50'),
        ("📺 Открыть YouTube", self.open_youtube, '#FF0000'),
        ("📁 Открыть проводник", self.open_explorer, '#2196F3'),
    ]

    for text, command, color in buttons_row1:
        btn = tk.Button(row1, text=text, command=command,
                        bg=color, fg='white', font=("Arial", 11, "bold"),
                        padx=20, pady=10, relief='raised', bd=2,
                        cursor='hand2')
        btn.pack(side='left', padx=10)

    row2 = tk.Frame(button_frame, bg=self.bg_color)
    row2.pack(pady=10)

    buttons_row2 = [
        ("⏻ Выключить компьютер", self.shutdown_computer, '#F44336'),
        ("🔄 Перезагрузить компьютер", self.restart_computer, '#FF9800'),
```

```
        (" 🚪 Выход из программы", self.exit_app, '#9E9E9E'),
    ]

    for text, command, color in buttons_row2:
        btn = tk.Button(row2, text=text, command=command,
                        bg=color, fg='white', font=("Arial", 11, "bold"),
                        padx=20, pady=10, relief='raised', bd=2,
                        cursor='hand2')
        btn.pack(side='left', padx=10)

    row3 = tk.Frame(button_frame, bg=self.bg_color)
    row3.pack(pady=20)

    buttons_row3 = [
        (" 📝 Блокнот", self.open_notepad, '#9C27B0'),
        (" 🧮 Калькулятор", self.open_calculator, '#009688'),
        (" 🐱 GitHub", self.open_github, '#333333'),
    ]

    for text, command, color in buttons_row3:
        btn = tk.Button(row3, text=text, command=command,
                        bg=color, fg='white', font=("Arial", 10),
                        padx=15, pady=8, relief='raised', bd=1,
                        cursor='hand2')
        btn.pack(side='left', padx=5)

def create_system_info(self):
    """Информация о системе"""
    info_frame = tk.Frame(self.root, bg=self.bg_color)
    info_frame.pack(pady=20)

    sys_info = f"Система: {platform.system()} {platform.release()}\n"
    sys_info += f"Python: {platform.python_version()}\n"
    sys_info += f"Процессор: {platform.processor()[:30]}..."
```

```
        tk.Label(info_frame, text=sys_info,
                 font=("Courier", 9),
                 bg=self.bg_color, fg='#aaaaaa',
                 justify='left').pack()

        status_frame = tk.Frame(self.root, bg='#1a1a1a', height=30)
        status_frame.pack(side='bottom', fill='x')
        status_frame.pack_propagate(False)

        tk.Label(status_frame, text="Готов к работе",
                 font=("Arial", 9),
                 bg='#1a1a1a', fg='#4CAF50').pack(side='left', padx=10)

        tk.Label(status_frame, text=f"Админ: {'Да' if self.is_admin() else
'Нет'}",
                 font=("Arial", 9),
                 bg='#1a1a1a', fg='#FF9800').pack(side='right', padx=10)

    def setup_bindings(self):
        """Настройка горячих клавиш"""
        self.root.bind('<Control-b>', lambda e: self.open_browser())
        self.root.bind('<Control-y>', lambda e: self.open_youtube())
        self.root.bind('<Control-q>', lambda e: self.exit_app())
        self.root.bind('<Escape>', lambda e: self.exit_app())

    def is_admin(self):
        """Проверка прав администратора"""
        try:
            if platform.system() == "Windows":
                import ctypes
                return ctypes.windll.shell32.IsUserAnAdmin() != 0
            else:
                return os.geteuid() == 0
```

```
    except:
        return False

def open_browser(self):
    """Открыть браузер"""
    try:
        webbrowser.open("https://www.google.com")
        self.show_status("Браузер открыт")
    except Exception as e:
        self.show_error(f"Ошибка: {e}")

def open_youtube(self):
    """Открыть YouTube"""
    try:
        webbrowser.open("https://www.youtube.com")
        self.show_status("YouTube открыт")
    except Exception as e:
        self.show_error(f"Ошибка: {e}")

def open_explorer(self):
    """Открыть проводник"""
    try:
        if platform.system() == "Windows":
            os.startfile(".")
        elif platform.system() == "Darwin":
            subprocess.Popen(["open", "."])
        else:
            subprocess.Popen(["xdg-open", "."])
        self.show_status("Проводник открыт")
    except Exception as e:
        self.show_error(f"Ошибка: {e}")

def shutdown_computer(self):
    """Выключить компьютер"""
```

```
    if not self.is_admin():
        self.show_warning("Требуются права администратора!")
        return

    if messagebox.askyesno("Подтверждение",
                           "Вы уверены, что хотите выключить компьютер?\n"
                           "Все несохраненные данные будут потеряны!"):
        try:
            if platform.system() == "Windows":
                os.system("shutdown /s /t 5")
                self.show_status("Компьютер выключится через 5 секунд")
                messagebox.showinfo("Выключение",
                                    "Компьютер будет выключен через 5 секунд")
            elif platform.system() == "Darwin":
                subprocess.Popen(["sudo", "shutdown", "-h", "now"])
            else:
                subprocess.Popen(["sudo", "shutdown", "-h", "now"])
        except Exception as e:
            self.show_error(f"Ошибка: {e}")

def restart_computer(self):
    """Перезагрузить компьютер"""
    if not self.is_admin():
        self.show_warning("Требуются права администратора!")
        return

    if messagebox.askyesno("Подтверждение",
                           "Вы уверены, что хотите перезагрузить компьютер?\n"
                           "Все несохраненные данные будут потеряны!"):
        try:
            if platform.system() == "Windows":
                os.system("shutdown /r /t 5")
                self.show_status("Перезагрузка через 5 секунд")
                messagebox.showinfo("Перезагрузка",
```

```
                "Компьютер перезагрузится через 5 секунд")
        elif platform.system() == "Darwin":
            subprocess.Popen(["sudo", "shutdown", "-r", "now"])
        else:
            subprocess.Popen(["sudo", "shutdown", "-r", "now"])
    except Exception as e:
        self.show_error(f"Ошибка: {e}")

def exit_app(self):
    """Выйти из программы"""
    if messagebox.askyesno("Выход", "Закрыть программу?"):
        self.root.destroy()

def open_notepad(self):
    """Открыть блокнот"""
    try:
        if platform.system() == "Windows":
            os.system("notepad")
        elif platform.system() == "Darwin":
            subprocess.Popen(["open", "-a", "TextEdit"])
        else:
            subprocess.Popen(["gedit"])
        self.show_status("Блокнот открыт")
    except Exception as e:
        self.show_error(f"Ошибка: {e}")

def open_calculator(self):
    """Открыть калькулятор"""
    try:
        if platform.system() == "Windows":
            os.system("calc")
        elif platform.system() == "Darwin":
            subprocess.Popen(["open", "-a", "Calculator"])
        else:
```

```
            subprocess.Popen(["gnome-calculator"])
        self.show_status("Калькулятор открыт")
    except Exception as e:
        self.show_error(f"Ошибка: {e}")

def open_github(self):
    """Открыть GitHub"""
    try:
        webbrowser.open("https://github.com")
        self.show_status("GitHub открыт")
    except Exception as e:
        self.show_error(f"Ошибка: {e}")


def show_status(self, message):
    """Показать статус"""
    print(f"[STATUS] {message}")

def show_error(self, message):
    """Показать ошибку"""
    print(f"[ERROR] {message}")
    messagebox.showerror("Ошибка", message)

def show_warning(self, message):
    """Показать предупреждение"""
    print(f"[WARNING] {message}")
    messagebox.showwarning("Предупреждение", message)
```

8.1.2. Вспомогательные функции (файл запуска)

```
def check_admin():
    """Проверка и запрос прав администратора"""
    if platform.system() == "Windows":
        try:
            import ctypes
            is_admin = ctypes.windll.shell32.IsUserAnAdmin() != 0
            if not is_admin:
                print("Запрашиваю права администратора...")
                ctypes.windll.shell32.ShellExecuteW(
                    None, "runas", sys.executable, " ".join(sys.argv), None, 1
                )
                return False
        except:
            pass
    return True

def main():
    """Главная функция"""
    if not check_admin():
        return
          root = tk.Tk()
    app = SystemControlPanel(root)

    root.update_idletasks()
    width = root.winfo_width()
    height = root.winfo_height()
    x = (root.winfo_screenwidth() // 2) - (width // 2)
    y = (root.winfo_screenheight() // 2) - (height // 2)
root.geometry(f'{width}x{height}+{x}+{y}')
    root.mainloop()

if __name__ == "__main__":main()
```

9. ТЕКСТЫ СИСТЕМОНЕЗАВИСИМОЙ ЧАСТИ

9.1. Модуль инициализации (файл main.py)

```
import tkinter as tk
import platform
import sys

def check_admin():
    """Проверка и запрос прав администратора"""
    if platform.system() == "Windows":
        try:
            import ctypes
            is_admin = ctypes.windll.shell32.IsUserAnAdmin() != 0
            if not is_admin:
                print("Запрашиваю права администратора...")
                ctypes.windll.shell32.ShellExecuteW(
                    None, "runas", sys.executable, " ".join(sys.argv), None, 1
                )
                return False
        except:
            pass
    return True

def main():
    """Главная функция запуска"""
    if not check_admin():
        return

    root = tk.Tk()
    app = SystemControlPanel(root)

    # Центрирование окна
    root.update_idletasks()
    width = root.winfo_width()
```

```
    height = root.winfo_height()
    x = (root.winfo_screenwidth() // 2) - (width // 2)
    y = (root.winfo_screenheight() // 2) - (height // 2)
    root.geometry(f'{width}x{height}+{x}+{y}')

    root.mainloop()

if __name__ == "__main__":
main()
```

9.2. Модуль главного класса (файл system_control.py)

```
import tkinter as tk
from tkinter import messagebox
import webbrowser
import os
import platform
import subprocess

class SystemControlPanel:
    def __init__(self, root):
        self.root = root
        self.setup_window()
        self.create_widgets()
        self.setup_bindings()

# ... (здесь весь код класса SystemControlPanel) ...
```

9.3. Модуль проверки прав (файл admin_check.py)

```
import platform
import os
import ctypes

def is_admin():
    """Проверка прав администратора (x10)"""
    try:
        if platform.system() == "Windows":
            return ctypes.windll.shell32.IsUserAnAdmin() != 0
        else:
            return os.geteuid() == 0
    except:
        return False
```

9.4. Модуль системных действий Windows (файл windows_actions.py)

```
import os
import subprocess

def open_explorer_windows():
    """Открыть проводник в Windows"""
    os.startfile(".")

def shutdown_windows():
    """Выключить компьютер в Windows"""
    os.system("shutdown /s /t 5")

def restart_windows():
    """Перезагрузить компьютер в Windows"""
    os.system("shutdown /r /t 5")

def open_notepad_windows():
    """Открыть блокнот в Windows"""
```

```
    os.system("notepad")

def open_calculator_windows():
    """Открыть калькулятор в Windows"""
os.system("calc")
```

9.5. Модуль системных действий macOS (файл mac_actions.py)

```
import subprocess

def open_explorer_mac():
    """Открыть Finder в macOS"""
    subprocess.Popen(["open", "."])

def shutdown_mac():
    """Выключить компьютер в macOS"""
    subprocess.Popen(["sudo", "shutdown", "-h", "now"])

def restart_mac():
    """Перезагрузить компьютер в macOS"""
    subprocess.Popen(["sudo", "shutdown", "-r", "now"])

def open_notepad_mac():
    """Открыть TextEdit в macOS"""
    subprocess.Popen(["open", "-a", "TextEdit"])

def open_calculator_mac():
    """Открыть калькулятор в macOS"""
subprocess.Popen(["open", "-a", "Calculator"])
```

9.6. Модуль системных действий Linux (файл linux_actions.py)

```
import subprocess

def open_explorer_linux():
```

```
    """Открыть файловый менеджер в Linux"""
    subprocess.Popen(["xdg-open", "."])

def shutdown_linux():
    """Выключить компьютер в Linux"""
    subprocess.Popen(["sudo", "shutdown", "-h", "now"])

def restart_linux():
    """Перезагрузить компьютер в Linux"""
    subprocess.Popen(["sudo", "shutdown", "-r", "now"])

def open_notepad_linux():
    """Открыть текстовый редактор в Linux"""
    subprocess.Popen(["gedit"])

def open_calculator_linux():
    """Открыть калькулятор в Linux"""
subprocess.Popen(["gnome-calculator"])
```

9.7. Модуль веб-действий (файл web_actions.py)

```
import webbrowser

def open_browser():
    """Открыть браузер с Google"""
    webbrowser.open("https://www.google.com")

def open_youtube():
    """Открыть YouTube"""
    webbrowser.open("https://www.youtube.com")

def open_github():
    """Открыть GitHub"""
webbrowser.open("https://github.com")
```

9.8. Модуль пользовательского интерфейса (файл ui_components.py)

```
import tkinter as tk

def create_status_bar(root, bg_color, text_color):
    """Создание статусной строки"""
    status_frame = tk.Frame(root, bg='#1a1a1a', height=30)
    status_frame.pack(side='bottom', fill='x')
    status_frame.pack_propagate(False)

    status_label = tk.Label(status_frame, text="Готов к работе",
                            font=("Arial", 9),
                            bg='#1a1a1a', fg='#4CAF50')
    status_label.pack(side='left', padx=10)

    return status_frame, status_label
```